LRU Cache - Complete LLD Interview Guide

Interview Duration: 45 minutes | Difficulty: Medium | Must-Know: ⭐⭐⭐

🎯 WHAT TO ACTUALLY WRITE IN INTERVIEW (20 mins coding)

✅ MUST WRITE ON WHITEBOARD/SCREEN:

1. Node Class (~3 mins)

```
class Node {
    int key;
    int value;
    Node prev;
    Node next;

    Node(int key, int value) {
        this.key = key;
        this.value = value;
    }
}
```

2. LRUCache Core Structure (~5 mins)

```
public class LRUCache {
    private HashMap<Integer, Node> cache;
    private int capacity;
    private Node head, tail;  // Dummy nodes

    public LRUCache(int capacity) {
        this.capacity = capacity;
        this.cache = new HashMap<>();

        // Dummy head and tail
        head = new Node(0, 0);
        tail = new Node(0, 0);
        head.next = tail;
        tail.prev = head;
    }
}
```

3. Core Operations - get() and put() (~12 mins)

```java
public int get(int key) {
    if (!cache.containsKey(key)) {
        return -1;
    }

    Node node = cache.get(key);
    moveToHead(node);  // Update access order
    return node.value;
}

public void put(int key, int value) {
    if (cache.containsKey(key)) {
        Node node = cache.get(key);
        node.value = value;
        moveToHead(node);
    } else {
        Node newNode = new Node(key, value);
        cache.put(key, newNode);
        addToHead(newNode);

        if (cache.size() > capacity) {
            Node tail = removeTail();
            cache.remove(tail.key);
        }
    }
}

// Helper methods - write signatures, explain logic
private void moveToHead(Node node) {
    removeNode(node);
    addToHead(node);
}

private void addToHead(Node node) {
    node.next = head.next;
    node.prev = head;
    head.next.prev = node;
    head.next = node;
}

private void removeNode(Node node) {
    node.prev.next = node.next;
    node.next.prev = node.prev;
}

private Node removeTail() {
    Node node = tail.prev;
    removeNode(node);
    return node;
}
```

🗣️ **EXPLAIN VERBALLY (Don't write full code):**

- "HashMap gives O(1) lookup, Doubly Linked List maintains access order"
- "Head = most recently used, Tail = least recently used"
- "Dummy head/tail simplify boundary conditions - no null checks"
- "For thread safety, add synchronized keyword to get/put methods"
- "For generics, change int to <K, V> generic types"
- "Can extend with TTL using timestamps and cleanup thread"

CONVERSATIONAL SCRIPT (How to approach in interview)

Phase 1: Requirements Clarification (3 mins)

You: "Let me clarify the requirements for the LRU Cache system."

Functional Requirements:

- "The cache should have a fixed capacity"
- "It should support get(key) and put(key, value) operations"
- "When the cache is full, it should evict the least recently used item"
- "Both get and put should be O(1) operations"
- "Should we support null keys or values?"

Interviewer: "No null keys. Null values are okay. Focus on O(1) operations."

You: "Got it. For non-functional requirements:"

- "Thread-safe operations for concurrent access"
- "Memory efficient implementation"
- "Should handle edge cases like negative capacity, duplicate keys"

Interviewer: "Yes, thread safety is important. Proceed with design."

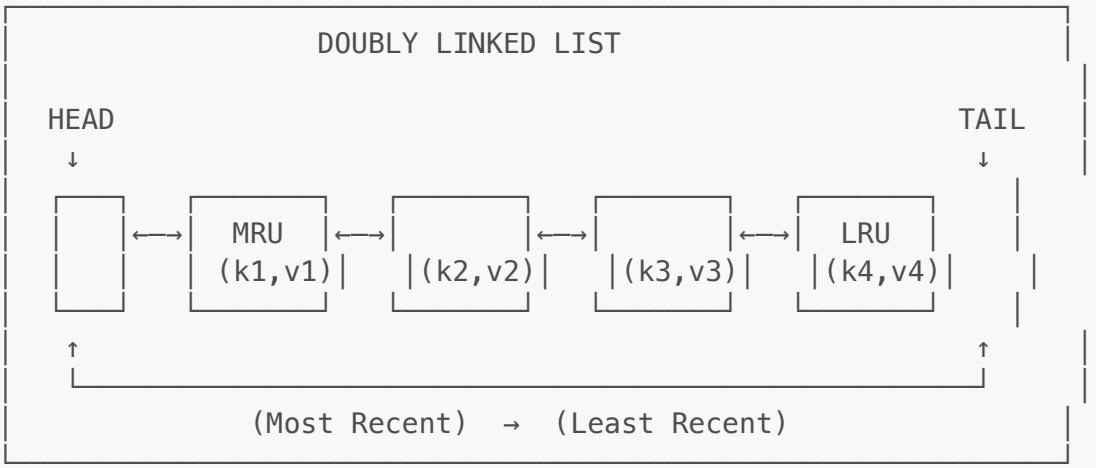
Phase 2: Core Approach (5 mins)

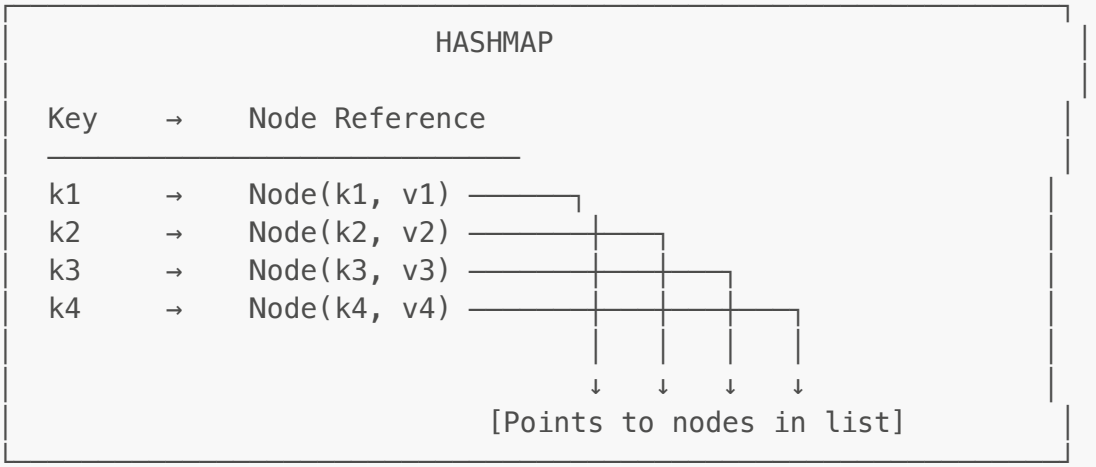
You: "For O(1) get and put operations with LRU eviction, I'll use a combination of two data structures:"

```
┌─────────────────────────────────────────────────────────┐
│              LRU CACHE ARCHITECTURE                     │
└─────────────────────────────────────────────────────────┘

Data Structures:
1. HashMap<K, Node<K, V>>  - O(1) access to any node
2. Doubly Linked List      - O(1) add/remove operations

Why this combination?
├─ HashMap: Fast key lookup
└─ Doubly Linked List: Maintains order (MRU to LRU)
```



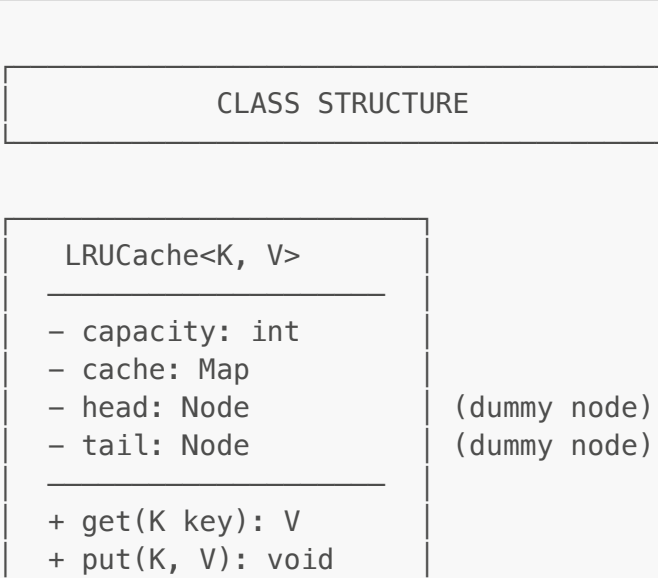


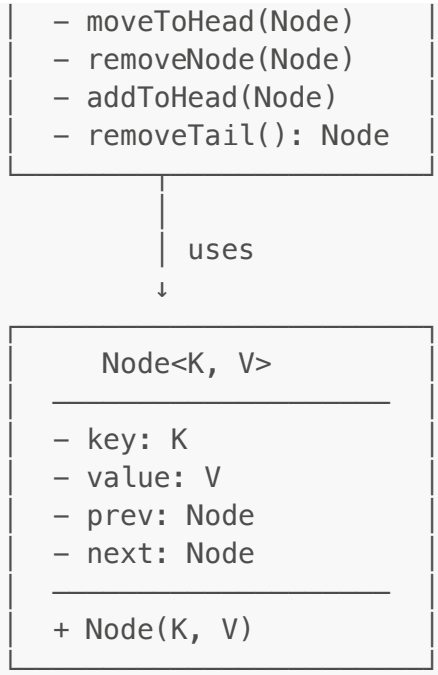
You: "The head always points to the most recently used item, and the tail points to the least recently used. When we access an item, we move it to the head. When we need to evict, we remove from the tail."

Interviewer: "Makes sense. Show me the implementation."

Phase 3: Class Design (5 mins)

You: "Let me design the classes:"





Phase 4: Core Implementation (20 mins)

You: "Here's the complete implementation:"

1. Node Class

```
public class Node<K, V> {
    K key;
    V value;
    Node<K, V> prev;
    Node<K, V> next;

    public Node(K key, V value) {
        this.key = key;
        this.value = value;
    }
}
```

You: "The Node stores the key-value pair and references to previous and next nodes in the doubly linked list."

---

2. LRUCache Class (Complete Implementation)

```
import java.util.HashMap;
import java.util.Map;

public class LRUCache<K, V> {
```

```
    private final int capacity;
    private final Map<K, Node<K, V>> cache;
    private final Node<K, V> head;  // Dummy head (most recent)
    private final Node<K, V> tail;  // Dummy tail (least recent)

    public LRUCache(int capacity) {
        if (capacity <= 0) {
            throw new IllegalArgumentException("Capacity must be
positive");
        }

        this.capacity = capacity;
        this.cache = new HashMap<>();

        // Initialize dummy nodes
        this.head = new Node<>(null, null);
        this.tail = new Node<>(null, null);
        head.next = tail;
        tail.prev = head;
    }

    /**
     * Get value for key
     * - If exists: move to head (mark as recently used) and return value
     * - If not exists: return null
     * Time Complexity: O(1)
     */
    public synchronized V get(K key) {
        Node<K, V> node = cache.get(key);

        if (node == null) {
            return null;
        }

        // Move accessed node to head (most recently used)
        moveToHead(node);
        return node.value;
    }

    /**
     * Put key-value pair
     * - If key exists: update value and move to head
     * - If key doesn't exist: create new node, add to head
     * - If cache is full: evict LRU (tail) before adding
     * Time Complexity: O(1)
     */
    public synchronized void put(K key, V value) {
        Node<K, V> node = cache.get(key);

        if (node != null) {
            // Key exists - update value and move to head
            node.value = value;
            moveToHead(node);
        } else {
```

```
            // New key - create new node
            Node<K, V> newNode = new Node<>(key, value);
            cache.put(key, newNode);
            addToHead(newNode);

            // Check if cache is full
            if (cache.size() > capacity) {
                // Evict LRU item (tail)
                Node<K, V> removed = removeTail();
                cache.remove(removed.key);
            }
        }
    }

    /**
     * Move existing node to head
     */
    private void moveToHead(Node<K, V> node) {
        removeNode(node);
        addToHead(node);
    }

    /**
     * Remove node from its current position
     */
    private void removeNode(Node<K, V> node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    /**
     * Add node right after head (most recently used position)
     */
    private void addToHead(Node<K, V> node) {
        node.next = head.next;
        node.prev = head;
        head.next.prev = node;
        head.next = node;
    }

    /**
     * Remove and return tail node (least recently used)
     */
    private Node<K, V> removeTail() {
        Node<K, V> lruNode = tail.prev;
        removeNode(lruNode);
        return lruNode;
    }

    /**
     * Get current size
     */
    public int size() {
        return cache.size();
```

```
    }

    /**
     * Check if cache is empty
     */
    public boolean isEmpty() {
        return cache.isEmpty();
    }

    /**
     * Clear the cache
     */
    public synchronized void clear() {
        cache.clear();
        head.next = tail;
        tail.prev = head;
    }

    /**
     * Display cache contents (for debugging)
     */
    public void display() {
        System.out.println("\n=== LRU Cache Contents ===");
        System.out.println("Capacity: " + capacity);
        System.out.println("Size: " + cache.size());
        System.out.print("Order (MRU → LRU): ");

        Node<K, V> current = head.next;
        while (current != tail) {
            System.out.print("[" + current.key + ":" + current.value + "]
");
            current = current.next;
        }
        System.out.println("\n========================\n");
    }
}
```

Phase 5: Step-by-Step Walkthrough (5 mins)

You: "Let me walk through a concrete example:"

```
Example: LRUCache with capacity = 3

Step 1: put(1, "A")
─────────────────────────────────────────
HashMap: {1 → Node(1,"A")}
List:    HEAD ←→ [1:A] ←→ TAIL

Step 2: put(2, "B")
─────────────────────────────────────────
```

```
HashMap: {1 → Node(1,"A"), 2 → Node(2,"B")}
List:    HEAD ←→ [2:B] ←→ [1:A] ←→ TAIL
                 ↑ MRU           ↑ LRU

Step 3: put(3, "C")
──────────────────────────────────────────────
HashMap: {1→Node(1,"A"), 2→Node(2,"B"), 3→Node(3,"C")}
List:    HEAD ←→ [3:C] ←→ [2:B] ←→ [1:A] ←→ TAIL
                 ↑ MRU                   ↑ LRU

Step 4: get(1) - Access existing key
──────────────────────────────────────────────
HashMap: {1→Node(1,"A"), 2→Node(2,"B"), 3→Node(3,"C")}
List:    HEAD ←→ [1:A] ←→ [3:C] ←→ [2:B] ←→ TAIL
                 ↑ MRU (moved)           ↑ LRU
Return: "A"

Step 5: put(4, "D") - Cache is full, evict LRU
──────────────────────────────────────────────
Evict: [2:B] (least recently used)
HashMap: {1→Node(1,"A"), 3→Node(3,"C"), 4→Node(4,"D")}
List:    HEAD ←→ [4:D] ←→ [1:A] ←→ [3:C] ←→ TAIL
                 ↑ MRU                   ↑ LRU

Step 6: put(1, "A_updated") - Update existing key
──────────────────────────────────────────────
HashMap: {1→Node(1,"A_updated"), 3→Node(3,"C"), 4→Node(4,"D")}
List:    HEAD ←→ [1:A_updated] ←→ [4:D] ←→ [3:C] ←→ TAIL
                 ↑ MRU (moved)                 ↑ LRU

Step 7: get(5) - Non-existent key
──────────────────────────────────────────────
Return: null
List unchanged
```

Phase 6: Usage Example (3 mins)

You: "Here's how to use the LRU Cache:"

```java
public class LRUCacheDemo {
    public static void main(String[] args) {
        // Create cache with capacity 3
        LRUCache<Integer, String> cache = new LRUCache<>(3);

        System.out.println("=== LRU Cache Demo ===\n");

        // Test 1: Add items
        System.out.println("Adding items...");
        cache.put(1, "Apple");
        cache.put(2, "Banana");
```

```
        cache.put(3, "Cherry");
        cache.display();

        // Test 2: Access existing item
        System.out.println("Accessing key 1...");
        String value = cache.get(1);
        System.out.println("Got: " + value);
        cache.display();

        // Test 3: Add item when full (eviction)
        System.out.println("Adding 4th item (cache full)...");
        cache.put(4, "Date");
        cache.display();
        // Key 2 should be evicted (LRU)

        // Test 4: Try to get evicted key
        System.out.println("Trying to get evicted key 2...");
        value = cache.get(2);
        System.out.println("Got: " + value); // null

        // Test 5: Update existing key
        System.out.println("\nUpdating key 1...");
        cache.put(1, "Avocado");
        cache.display();

        // Test 6: Thread safety test
        System.out.println("=== Thread Safety Test ===");
        LRUCache<Integer, Integer> threadSafeCache = new LRUCache<>(100);

        // Create multiple threads
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                threadSafeCache.put(i, i * 2);
            }
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                threadSafeCache.get(i);
            }
        });

        t1.start();
        t2.start();

        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Final cache size: " + threadSafeCache.size());
        System.out.println("Thread safety test passed!");
```

```
    }
}
```

Expected Output:

```
=== LRU Cache Demo ===

Adding items...

=== LRU Cache Contents ===
Capacity: 3
Size: 3
Order (MRU → LRU): [3:Cherry] [2:Banana] [1:Apple]
========================

Accessing key 1...
Got: Apple

=== LRU Cache Contents ===
Capacity: 3
Size: 3
Order (MRU → LRU): [1:Apple] [3:Cherry] [2:Banana]
========================

Adding 4th item (cache full)...

=== LRU Cache Contents ===
Capacity: 3
Size: 3
Order (MRU → LRU): [4:Date] [1:Apple] [3:Cherry]
========================

Trying to get evicted key 2...
Got: null

Updating key 1...

=== LRU Cache Contents ===
Capacity: 3
Size: 3
Order (MRU → LRU): [1:Avocado] [4:Date] [3:Cherry]
========================
```

Phase 7: Advanced Features (5 mins)

You: "Let me show some advanced features we can add:"

1. TTL (Time-To-Live) Support

```
public class LRUCacheWithTTL<K, V> extends LRUCache<K, V> {
    private final Map<K, Long> expirationMap;
    private final long ttlMillis;

    public LRUCacheWithTTL(int capacity, long ttlMillis) {
        super(capacity);
        this.expirationMap = new HashMap<>();
        this.ttlMillis = ttlMillis;
    }

    @Override
    public synchronized V get(K key) {
        if (isExpired(key)) {
            // Remove expired entry
            cache.remove(key);
            expirationMap.remove(key);
            return null;
        }
        return super.get(key);
    }

    @Override
    public synchronized void put(K key, V value) {
        super.put(key, value);
        expirationMap.put(key, System.currentTimeMillis() + ttlMillis);
    }

    private boolean isExpired(K key) {
        Long expiration = expirationMap.get(key);
        return expiration != null && System.currentTimeMillis() >
expiration;
    }
}
```

2. Statistics Tracking

```
public class LRUCacheWithStats<K, V> extends LRUCache<K, V> {
    private long hitCount = 0;
    private long missCount = 0;

    public LRUCacheWithStats(int capacity) {
        super(capacity);
    }

    @Override
    public synchronized V get(K key) {
        V value = super.get(key);
        if (value != null) {
            hitCount++;
        } else {
```

```
            missCount++;
        }
        return value;
    }

    public synchronized double getHitRate() {
        long total = hitCount + missCount;
        return total == 0 ? 0.0 : (double) hitCount / total;
    }

    public synchronized void printStats() {
        System.out.println("=== Cache Statistics ===");
        System.out.println("Hits: " + hitCount);
        System.out.println("Misses: " + missCount);
        System.out.println("Hit Rate: " + String.format("%.2f%%",
getHitRate() * 100));
        System.out.println("======================");
    }
}
```

3. LRU with Size-based Eviction

```
public class LRUCacheWithSize<K, V> {
    private final long maxSizeBytes;
    private long currentSize = 0;
    private final Map<K, CacheEntry<K, V>> cache;
    // ... similar implementation but track size

    private static class CacheEntry<K, V> {
        K key;
        V value;
        long size;  // Size in bytes
        CacheEntry<K, V> prev;
        CacheEntry<K, V> next;
    }

    // Evict based on total size instead of count
}
```

Phase 8: Edge Cases & Thread Safety (3 mins)

You: "Let me discuss important edge cases:"

1. Concurrent Access:

```
// All methods are synchronized for thread safety
public synchronized V get(K key) { ... }
public synchronized void put(K key, V value) { ... }
```

```
// Alternative: Use ReentrantReadWriteLock for better performance
private final ReadWriteLock lock = new ReentrantReadWriteLock();

public V get(K key) {
    lock.readLock().lock();
    try {
        // ... get logic
    } finally {
        lock.readLock().unlock();
    }
}

public void put(K key, V value) {
    lock.writeLock().lock();
    try {
        // ... put logic
    } finally {
        lock.writeLock().unlock();
    }
}
```

2. Capacity Edge Cases:

```
// Capacity = 0 or negative
public LRUCache(int capacity) {
    if (capacity <= 0) {
        throw new IllegalArgumentException("Capacity must be positive");
    }
    // ...
}

// Capacity = 1 (special case)
LRUCache<Integer, String> cache = new LRUCache<>(1);
cache.put(1, "A");  // [1:A]
cache.put(2, "B");  // [2:B], evicts 1
cache.get(1);       // null
```

3. Null Handling:

```
public synchronized void put(K key, V value) {
    if (key == null) {
        throw new IllegalArgumentException("Key cannot be null");
    }
    // value can be null (allowed in this implementation)
    // ...
}
```

Phase 9: Follow-up Questions & Answers

Interviewer: "How would you implement an LFU (Least Frequently Used) cache instead?"

You:

```
public class LFUCache<K, V> {
    private final Map<K, Node<K, V>> cache;
    private final Map<Integer, DoublyLinkedList<K, V>> frequencyMap;
    private int minFrequency;
    private final int capacity;

    private static class Node<K, V> {
        K key;
        V value;
        int frequency;
    }

    // Similar structure but track frequency
    // Evict item with lowest frequency
}
```

Interviewer: "What if we need to support both LRU and expiration?"

You: "I showed the TTL extension earlier. We can combine both:"

```
// Check expiration on every get
// Clean up expired entries periodically
// Evict LRU when capacity is reached
```

Interviewer: "How would you distribute this cache across multiple servers?"

You:

```
// Use consistent hashing
public class DistributedLRUCache<K, V> {
    private final ConsistentHash<String, LRUCache<K, V>> ring;

    public V get(K key) {
        LRUCache<K, V> node = ring.get(key.toString());
        return node.get(key);
    }

    // Handle node failures with replication
}
```

KEY TAKEAWAYS

Data Structures:

✅ **HashMap** - O(1) key lookup ✅ **Doubly Linked List** - O(1) add/remove, maintains order ✅ **Dummy Head/Tail** - Simplifies boundary conditions

Time Complexity:

Operation	Complexity	Explanation
get()	O(1)	HashMap lookup + List reorder
put()	O(1)	HashMap insert + List operations
evict()	O(1)	Remove tail node

Space Complexity:

- O(n) where n = capacity
- HashMap: O(n)
- Linked List: O(n)

Thread Safety Options:

1. **Synchronized methods** - Simple, works for low concurrency
2. **ReentrantReadWriteLock** - Better for read-heavy workloads
3. **ConcurrentHashMap** - More complex, highest performance

Design Principles:

✅ **Single Responsibility** - Node handles data, LRUCache handles logic ✅ **Encapsulation** - Private helper methods ✅ **Generics** - Works with any key-value types

---

SOLID PRINCIPLES IN DEPTH

You: "Let me explain how SOLID principles apply to the LRU Cache design. This is important for understanding good OOP design."

---

1. Single Responsibility Principle (SRP)

Purpose: Each class should have only ONE reason to change.

Problem it solves: Without SRP, the cache becomes a monolithic mess:

```
// BAD: Cache doing too much
class LRUCache {
    // Cache logic
    public void get() { ... }
```

```
    public void put() { ... }

    // Statistics tracking
    public void recordHit() { ... }
    public void recordMiss() { ... }

    // Persistence
    public void saveToDisk() { ... }
    public void loadFromDisk() { ... }

    // Monitoring
    public void sendMetrics() { ... }
}
// Too many responsibilities! Any change affects the whole class.
```

Advantages:

- ✅ **Clear purpose** - Each class does one thing well
- ✅ **Easy to test** - Test one responsibility in isolation
- ✅ **Easy to maintain** - Changes are localized
- ✅ **Easy to understand** - Small, focused classes

In our design:

```
// GOOD: Separated responsibilities

// Node: ONLY stores data and pointers
class Node {
    int key, value;
    Node prev, next;
}

// LRUCache: ONLY manages cache operations (get, put, evict)
class LRUCache {
    private HashMap<Integer, Node> cache;
    private Node head, tail;

    public int get(int key) { ... }
    public void put(int key, int value) { ... }
}

// CacheStatistics: ONLY tracks metrics (separate class)
class CacheStatistics {
    private int hits, misses;
    public void recordHit() { ... }
    public double getHitRate() { ... }
}

// CachePersistence: ONLY handles disk I/O (separate class)
class CachePersistence {
    public void save(LRUCache cache) { ... }
```

```
    public LRUCache load() { ... }
}
```

Interview tip: "The Node class only stores data. The LRUCache class only manages cache operations. If I need to add persistence, I create a separate `CachePersistence` class. Each class has one job."

---

2. Open/Closed Principle (OCP)

Purpose: Classes should be OPEN for extension but CLOSED for modification.

Problem it solves: Without OCP, adding features requires modifying existing code:

```
// BAD: Hard-coded eviction policy
class Cache {
    public void evict() {
        // LRU logic hard-coded here
        Node lru = tail.prev;
        removeNode(lru);

        // To add LFU, you must MODIFY this method - RISKY!
    }
}
```

Advantages:

- ✅ **No regression risk** - Existing code stays untouched
- ✅ **Easy to extend** - Add new policies by adding classes
- ✅ **Multiple policies** - Switch at runtime
- ✅ **Stable core** - Cache logic never changes

In our design:

```
// GOOD: Policy-based design (Strategy pattern)

interface EvictionPolicy {
    void recordAccess(int key);
    int evict();
}

class LRUEvictionPolicy implements EvictionPolicy {
    private LinkedHashMap<Integer, Node> orderMap;

    @Override
    public int evict() {
        return orderMap.keySet().iterator().next();  // Oldest
    }
}
```

```
class LFUEvictionPolicy implements EvictionPolicy {
    private Map<Integer, Integer> frequencyMap;

    @Override
    public int evict() {
        // Return key with lowest frequency
    }
}

class MRUEvictionPolicy implements EvictionPolicy {
    @Override
    public int evict() {
        // Return most recently used
    }
}

class Cache {
    private EvictionPolicy policy;

    public Cache(EvictionPolicy policy) {
        this.policy = policy;  // Inject any policy!
    }

    public void evict() {
        int keyToEvict = policy.evict();  // Uses interface
        // Remove from cache
    }
}

// Usage:
Cache lruCache = new Cache(new LRUEvictionPolicy());
Cache lfuCache = new Cache(new LFUEvictionPolicy());  // NEW - zero changes
to Cache!
```

Interview tip: "To add LFU eviction, I create `LFUEvictionPolicy` implementing the interface. Zero changes to the core Cache class. The system is closed for modification but open for extension."

---

3. Liskov Substitution Principle (LSP)

Purpose: Subclasses must be substitutable for their parent classes without breaking behavior.

Problem it solves: Without LSP, derived classes violate base class contracts:

```
// BAD: Violates LSP
class Cache {
    public int get(int key) {
        // Contract: Returns value or -1 if not found
        return map.getOrDefault(key, -1);
    }
}
```

```
class ReadOnlyCache extends Cache {
    @Override
    public int get(int key) {
        throw new UnsupportedOperationException("Read-only!");  // BREAKS
CONTRACT!
    }
}

// Code expecting Cache behavior will crash:
Cache cache = new ReadOnlyCache();
int val = cache.get(5);  // BOOM! Exception instead of -1
```

Advantages:

- ✅ **Predictable behavior** - Subclasses work as expected
- ✅ **Polymorphism works** - Can use base type everywhere
- ✅ **Code reuse** - Write once, works for all subtypes
- ✅ **No surprises** - Substitution doesn't break code

In our design:

```
// GOOD: All eviction policies honor the contract

interface EvictionPolicy {
    void recordAccess(int key);   // Contract: Record that key was accessed
    int evict();                  // Contract: Return key to evict
}

class LRUEvictionPolicy implements EvictionPolicy {
    @Override
    public void recordAccess(int key) { /* Move to head */ }  // ✓ Honors
contract

    @Override
    public int evict() { return tailKey; }  // ✓ Returns a key, as promised
}

class LFUEvictionPolicy implements EvictionPolicy {
    @Override
    public void recordAccess(int key) { /* Increment frequency */ }  // ✓
Honors contract

    @Override
    public int evict() { return lowestFreqKey; }  // ✓ Returns a key, as
promised
}

// Polymorphism works perfectly:
EvictionPolicy policy = new LRUEvictionPolicy();  // Or LFUEvictionPolicy
```

```
policy.recordAccess(5);  // Works for ANY policy
int keyToEvict = policy.evict();  // Works for ANY policy
```

Interview tip: "Any code that works with `EvictionPolicy` will work with `LRUEvictionPolicy`, `LFUEvictionPolicy`, or any future policy. They all honor the contract - `evict()` always returns a valid key."

---

4. Interface Segregation Principle (ISP)

Purpose: Clients should not be forced to depend on interfaces they don't use.

Problem it solves: Without ISP, interfaces force unnecessary dependencies:

```
// BAD: Fat interface forces implementations of unused methods
interface Cache {
    int get(int key);
    void put(int key, int value);
    void remove(int key);
    void clear();
    int size();
    boolean isEmpty();
    void save();              // Not all caches need persistence
    void load();              // Not all caches need persistence
    void enableTTL();         // Not all caches need TTL
    void setMaxMemory();      // Not all caches need memory limits
}

// Simple in-memory cache must implement ALL methods!
class SimpleCache implements Cache {
    @Override
    public void save() { throw new UnsupportedOperationException(); }  //
Forced!
    @Override
    public void load() { throw new UnsupportedOperationException(); }  //
Forced!
}
```

Advantages:

- ✅ **Lean interfaces** - Only necessary methods
- ✅ **Better cohesion** - Related methods together
- ✅ **No dummy code** - No forced implementations
- ✅ **Decoupled** - Changes don't ripple

In our design:

```
// GOOD: Segregated interfaces
```

```java
// Core cache operations
interface Cache<K, V> {
    V get(K key);
    void put(K key, V value);
}

// Optional: Statistics (separate interface)
interface CacheStatistics {
    int getHits();
    int getMisses();
    double getHitRate();
}

// Optional: Persistence (separate interface)
interface Persistable {
    void save(String path);
    void load(String path);
}

// Optional: Expiration (separate interface)
interface Expirable {
    void setTTL(int key, long ttl);
    void evictExpired();
}

// Implement only what you need:
class SimpleLRUCache implements Cache<Integer, Integer> {
    // Only implements get() and put() - nothing else!
}

class PersistentLRUCache implements Cache<Integer, Integer>, Persistable {
    // Implements cache + persistence, but NOT statistics
}

class FullFeaturedCache implements Cache<Integer, Integer>,
                                   CacheStatistics,
                                   Persistable,
                                   Expirable {
    // Implements everything - by choice, not force
}
```

Interview tip: "I keep interfaces focused. Core cache has only `get()` and `put()`. If you need persistence, implement `Persistable`. If you need stats, implement `CacheStatistics`. Clients depend only on what they need."

---

5. Dependency Inversion Principle (DIP)

Purpose: High-level modules should not depend on low-level modules. Both should depend on abstractions.

Problem it solves: Without DIP, high-level code is tightly coupled:

```
// BAD: Tight coupling to concrete implementation
class CacheManager {
    private LRUCache cache = new LRUCache(100);  // TIGHT COUPLING!

    public int getData(int key) {
        return cache.get(key);
        // If you want to switch to LFU, you must modify CacheManager!
    }
}
```

Advantages:

- ✅ **Loose coupling** - Easy to swap implementations
- ✅ **Testability** - Can inject mocks
- ✅ **Flexibility** - Change behavior at runtime
- ✅ **Maintainability** - Low-level changes don't affect high-level

In our design:

```
// GOOD: Depend on abstraction (interface)

interface Cache<K, V> {
    V get(K key);
    void put(K key, V value);
}

class LRUCache implements Cache<Integer, Integer> { ... }
class LFUCache implements Cache<Integer, Integer> { ... }
class FIFOCache implements Cache<Integer, Integer> { ... }

class CacheManager {
    private Cache<Integer, Integer> cache;  // Interface, not concrete
class!

    // Dependency Injection via constructor
    public CacheManager(Cache<Integer, Integer> cache) {
        this.cache = cache;
    }

    public int getData(int key) {
        return cache.get(key);  // Don't care about implementation!
    }
}

// At runtime, inject any implementation:
CacheManager manager1 = new CacheManager(new LRUCache(100));
CacheManager manager2 = new CacheManager(new LFUCache(100));  // Different
impl, same interface
```

```
// For testing, inject mock:
CacheManager testManager = new CacheManager(new MockCache());
```

Interview tip: "CacheManager doesn't know if it's using LRU or LFU - it just calls `get()` and `put()` on the interface. I can swap implementations at runtime. For testing, I inject a mock cache that returns predictable values."

KEY TAKEAWAYS

Design Patterns Used:

✅ **Custom Data Structure** - HashMap + Doubly Linked List ✅ **Strategy Pattern** - Different eviction policies (LRU, LFU, MRU) ✅ **Encapsulation** - Private helper methods ✅ **Generics** - Works with any key-value types

SOLID Principles Applied:

✅ **Single Responsibility (SRP)** - Node stores data, LRUCache manages operations, Statistics tracks metrics ✅ **Open/Closed (OCP)** - Add new eviction policies without modifying cache core ✅ **Liskov Substitution (LSP)** - All EvictionPolicy implementations are interchangeable ✅ **Interface Segregation (ISP)** - Separate interfaces for Cache, Statistics, Persistence, Expiration ✅ **Dependency Inversion (DIP)** - CacheManager depends on Cache interface, not concrete LRUCache

❌ Forgetting to update HashMap when moving nodes ❌ Not handling capacity = 0 or negative ❌ Removing from list but forgetting HashMap (memory leak) ❌ Not thread-safe in concurrent environment ❌ Forgetting dummy nodes (complex boundary handling) ❌ Incorrect order: evict → add vs add → evict ❌ Not updating node value when key exists

---

VARIATIONS YOU SHOULD KNOW

1. LFU Cache (Least Frequently Used)

- Evict item with lowest access count
- Use frequency counter

2. MRU Cache (Most Recently Used)

- Opposite of LRU
- Evict most recently used item

3. Segmented LRU (S-LRU)

- Divide into probationary and protected segments
- Better hit rate for some workloads

4. Time-aware LRU (TLRU)

- Consider both recency and time-to-live

- Used in CDNs

REAL-WORLD APPLICATIONS

✅ **Database Query Caching** - MySQL, PostgreSQL ✅ **Web Browser Cache** - Chrome, Firefox ✅ **CDN Edge Caching** - CloudFlare, Akamai ✅ **CPU Cache** - L1, L2, L3 caches ✅ **DNS Caching** - Recursive DNS servers ✅ **API Response Caching** - REST APIs ✅ **Session Storage** - Web applications

---

END OF LRU CACHE GUIDE

This is a **fundamental data structure** question asked by almost every company!